Design Patterns for Handling Data 2

Welcome back to CS 2100!

Prof. Rasika Bhalerao

Argsort

Numpy's `argsort()` takes a list and returns a list of its indices, moved to the locations they would be in if the list was sorted.

```
words: list[str] = 'never gonna give you up'.split()

sorted_alphabetically: list[str] = sorted(words)

print(sorted_alphabetically)  # ['give', 'gonna', 'never', 'up', 'you']

indices_of_sorted_words = np.argsort(words)

print(indices_of_sorted_words)  # [2 1 0 4 3]
```

It's saying if the list of words was sorted alphabetically, the word at index 2 would be first, then the word at index 1, then the word at index 0, etc. until the word at index 3.

Poll: What is output?

```
scores = np.array([72, 95, 58, 88])
order = np.argsort(scores)
print(order)
```

1. [58, 72, 88, 95]
2. [2, 0, 3, 1]
3. [3, 2, 0, 1]
4. [1, 3, 0, 2]

Motivating example

Grades come from two dataframes:

Pawtograder:

Student ID	HW1 score
001	40
002	39

Canvas:

Student ID	Number of lectures attended
001	10
002	11

How can we combine them into a single gradebook?

Combining dataframes

We previously discussed concatenating dataframes to add one to the "end" of the other.

```
df = pd.DataFrame(
    {'Person': ['Elephant', 'Cat'],
    'Age': [13, 10]})

new_rows = pd.DataFrame({
    'Person': ['Dog', 'Giraffe'],
    'Age': [3, 6]})

df = pd.concat([df, new_rows], ignore_index=True)
```

```
     Person  Age
0  Elephant   13
1       Cat   10
2       Dog    3
3   Giraffe    6
```

But that wouldn't work for our case (combining grade tables)

```python
import pandas as pd

df_hw = pd.DataFrame({
    'Student ID': ['001', '002'],
    'HW1 score': [40, 39]
})

df_canvas = pd.DataFrame({
    'Student ID': ['001', '002'],
    'N lec att': [10, 11]
})

df = pd.concat(
    [df_hw, df_canvas],
    ignore_index=True)
print(df)
```

```
  Student ID  HW1 score  N lec att
0        001       40.0        NaN
1        002       39.0        NaN
2        001        NaN       10.0
3        002        NaN       11.0
```

Merge: combining dataframes side-by-side instead of at the end

Default: `pandas.merge()` combines dataframes using any columns with the same name:

```
df = pd.DataFrame(
    {'Person': ['Elephant', 'Cat', 'Dog', 'Giraffe'],
    'Age': [13, 10, 3, 6]})

other_df = pd.DataFrame(
    {'Person': ['Dog', 'Giraffe', 'Elephant', 'Cat'],
     'BFF': ['Cat', 'Elephant', 'Giraffe', 'Cat']}
)

print(pd.merge(df, other_df))
```

```
     Person  Age       BFF
0  Elephant   13   Giraffe
1       Cat   10       Cat
2       Dog    3       Cat
3   Giraffe    6  Elephant
```

`merge()` works for our grade dataframe

```
df = pd.merge(df_hw, df_canvas)
```

```
  Student ID  HW1 score  Number of lectures attended
0        001         40                           10
1        002         39                           11
```

Next complication: What if both tables have Student ID and Name?

Pandas can't assume that we do something silly like this:

Pawtograder:

Student ID	Name	HW1 score
001	Mini	40
002	Micro	39

Canvas:

Student ID	Name	Number lectures attended
001	Giant	10
002	Micro	11

Which identifier will it use for merging?

Merge: combining dataframes side-by-side

Multiple columns with the same name -> specify which one to use with the `on` arg

If the columns with the same name don't "agree" across the two dataframes (e.g., in one dataset, the cat's age is 10, and in the other, the cat's age is 6), it will include both of the disagreeing columns, suffixed by the original dataframe (`x` or `y`):

```
df = pd.DataFrame(
    {'Person': ['Elephant', 'Cat', 'Dog', 'Giraffe'],
    'Age': [13, 10, 3, 6]})

other_df = pd.DataFrame(
    {'Person': ['Dog', 'Giraffe', 'Elephant', 'Cat'],
     'Age': [13, 10, 3, 6],  # these ages are different from the ones in df
     'BFF': ['Cat', 'Elephant', 'Giraffe', 'Cat']}
)

print(pd.merge(df, other_df, on='Person'))
```

Merge: combining dataframes side-by-side

If the columns with the same name don't "agree" across the two dataframes (e.g., in one dataset, the cat's age is 10, and in the other, the cat's age is 6), it will include both of the disagreeing columns, suffixed by the original dataframe (`x` or `y`):

```
     Person  Age_x  Age_y       BFF
0  Elephant     13      3   Giraffe
1       Cat     10      6       Cat
2       Dog      3     13       Cat
3   Giraffe      6     10  Elephant
```

Merge

Values that only appear in one dataset and not the other -> it will omit those rows entirely:

```
df = pd.DataFrame(
    {'Person': ['Elephant', 'Cat', 'Dog', 'Giraffe'],
    'Age': [13, 10, 3, 6]})

df_age_year = pd.DataFrame({
    'Age': [3, 6, 9, 10],
    'Year': [2022, 2019, 2016, 2015]
})

print(pd.merge(df, df_age_year))
```

```
    Person  Age  Year
0      Cat   10  2015
1      Dog    3  2022
2  Giraffe    6  2019
```

Merge

Don't like that strategy for values that only appear in one dataset? Use the `how` arg:

- `'left'` -> resulting dataframe will include all rows from the left dataframe, filling in `NaN` ("Not a Number") values for pieces missing on the right

```
df = pd.DataFrame(
    {'Person': ['Elephant', 'Cat', 'Dog', 'Giraffe'], 'Age': [13, 10, 3, 6]})

df_age_year = pd.DataFrame({
    'Age': [3, 6, 9, 10], 'Year': [2022, 2019, 2016, 2015]})

print(pd.merge(df, df_age_year, on='left'))
```

```
     Person  Age    Year
0  Elephant   13     NaN
1       Cat   10  2015.0
2       Dog    3  2022.0
3   Giraffe    6  2019.0
```

Merge

- `'left'` -> resulting dataframe will include all rows from the left dataframe, filling in `NaN` ("Not a Number") values for pieces missing on the right
- `'right'` -> result will include all rows from the right dataframe, filling in `NaN` for the pieces missing on the left

```
df = pd.DataFrame(
    {'Person': ['Elephant', 'Cat', 'Dog', 'Giraffe'], 'Age': [13, 10, 3, 6]})
df_age_year = pd.DataFrame({
    'Age': [3, 6, 9, 10], 'Year': [2022, 2019, 2016, 2015]})

print(pd.merge(df, df_age_year, on='right'))
```

```
    Person  Age  Year
0      Dog    3  2022
1  Giraffe    6  2019
2      NaN    9  2016
3      Cat   10  2015
```

Options for `merge()`'s `how` arg

- `'left'` -> resulting dataframe will include all rows from the left dataframe, filling in `NaN` ("Not a Number") values for pieces missing on the right
- `'right'` -> result will include all rows from the right dataframe, filling in `NaN` for the pieces missing on the left
- `'outer'` -> include all rows from both
- `'inner'` -> only include rows that were in both

```
df = pd.DataFrame(
    {'Person': ['Elephant', 'Cat', 'Dog', 'Giraffe'],
    'Age': [13, 10, 3, 6]})

df_age_year = pd.DataFrame({
    'Age': [3, 6, 9, 10],
    'Year': [2022, 2019, 2016, 2015]
})

print(pd.merge(df, df_age_year, on='outer'))
```

```
     Person  Age    Year
0  Elephant   13     NaN
1       Cat   10  2015.0
2       Dog    3  2022.0
3   Giraffe    6  2019.0
4       NaN    9  2016.0
```

Poll: What is output?

```
df_all_students = pd.DataFrame({
    'ID': [1, 2, 3, 4],
    'Name': ['Cat', 'Dog', 'Elephant', 'Giraffe']
})

df_swimming_class_grades = pd.DataFrame({
    'ID': [1, 3, 5],
    'Grade': ['A', 'B', 'C']
})

print(pd.merge(df_all_students, df_swimming_class_grades, on='ID', how='left'))
```

1. A dataframe with 3 rows
2. A dataframe with 4 rows
3. A dataframe with 5 rows
4. Error -- cannot merge dataframes with different lengths

Exercise

Assume we have two dataframes:

Prices:

Item	Cost
Blue Shirt	10
Black Shirt	12
White Shirt	12
Pink Shirt	10

Mini's purchases:

Item
White Shirt
White Shirt
Pink Shirt

How can we find the total cost of all of Mini's purchases?

Groupby motivation: Who is the top scorer in each course?

```
scores = pd.DataFrame({
    "course":  ["2100", "2100", "2100", "3100", "3100"],
    "student": ["Ana",  "Ben",  "Cy",   "Dee",  "Eli"],
    "score":   [88,     95,     79,     91,     84],
})

for c in scores["course"].unique():
    sub = scores[scores["course"] == c]
    print(c, sub.loc[sub["score"].idxmax(), "student"])
```

Groupby motivation: Who is the top scorer in each course?

```
scores = pd.DataFrame({
    "course":  ["2100", "2100", "2100", "3100", "3100"],
    "student": ["Ana",  "Ben",  "Cy",   "Dee",  "Eli"],
    "score":   [88,     95,     79,     91,     84],
})

scores.loc[scores.groupby("course")["score"].idxmax()]
```

course	student	score
2100	Ben	95
3100	Dee	91

`df["score"].max()` can't do this: it sees the whole column at once and returns 95, with no notion of "per course."

pandas `groupby`: Split → Apply → Combine

1. **Split**: divide rows into groups by the values in a key column
2. **Apply**: run a function on each group independently
3. **Combine**: stitch the per-group results into one object

`df.groupby("dept")["enroll"].sum()`

dept	course	enroll
CS	2100	120
CS	3100	90
Math	1341	60
Math	2331	45
Math	2310	30

dept	enroll
CS	210
Math	135

`df.groupby("dept")` alone computes nothing. It just describes the groups.
The work happens when you call an aggregation like `.sum()`.

Three Ways to "Apply"

Aggregate: one row per group

```
df.groupby("dept")["enroll"].agg(["mean", "max"])
```

Transform: same shape as the original, aligned to the original rows

```
df["share"] = df["enroll"] / df.groupby("dept")["enroll"].transform("sum")
```

Filter: keep or drop whole groups

```
df.groupby("dept").filter(lambda g: g["enroll"].sum() > 150)
```

Watch out for:

- `count` skips NaN, `size` counts all rows
- Rows with a NaN key are dropped unless `dropna=False`
- Multiple keys give a MultiIndex; use `.reset_index()` to flatten

dept	course	enroll
CS	2100	120
CS	3100	90
Math	1341	60
Math	2331	45
Math	2310	30

Poll: What does `df.groupby("dept")["enroll"].mean()` return?

1. A Series: CS → 105, Math → 45
2. A single number: 69
3. A DataFrame with 5 rows, one per course
4. Nothing is computed; it returns a lazy object

dept	course	enroll
CS	2100	120
CS	3100	90
Math	1341	60

Poll: We want a new column showing each course's enrollment as a fraction of its department's total enrollment. Which expression gives a result we can assign directly as a new column of `df`?

1. df.groupby("dept")["enroll"].sum()
2. df.groupby("dept")["enroll"].transform("sum")
3. df.groupby("dept").filter(lambda g: g["enroll"].sum() > 0)
4. df.groupby("dept")["enroll"].size()

Poll:

1. What is your main takeaway from today?

2. What would you like to revisit next time?